

Trabajo Práctico de Implementación

Secreto Compartido en Imágenes con Esteganografía

Criptografía y Seguridad (72.04) — 2026

Grupo 5

7 de junio de 2026

Resumen

Se implementó en lenguaje C el esquema de secreto compartido en imágenes propuesto por Wu y Lo [1], que adapta la construcción de Thien y Lin [2] reemplazando GF(251) por GF(257) y la permutación por una máscara XOR generada por un LCG de 48 bits con semilla de 2 bytes. Las sombras polinómicas resultantes se ocultan en portadoras BMP de 8 bpp mediante *LSB replacement*. El programa soporta $2 \leq k \leq 10$ y maneja los casos $k = 8$ y $k \neq 8$ con criterios diferenciados de tamaño de portadora. Este informe responde, en orden, las ocho cuestiones planteadas en la sección 5 de la consigna.

1. Análisis del documento de Wu y Lo

Consigna 5.1. *Discutir organización formal, descripción del algoritmo y notación del paper de Wu y Lo.*

Organización formal

El paper se organiza en siete partes a lo largo de cinco páginas: *Abstract*, *Introduction*, *Review* (con subsecciones sobre Shamir y Thien-Lin), *The Proposed Method*, *Experimental Results*, *Conclusions* y *References*. La estructura es eficiente y construye sobre Thien-Lin [2], lo cual abrevia la exposición pero hace al texto **no autocontenido**: un lector que no tenga el paper de Thien-Lin a mano se pierde varios detalles implícitos — en particular el porqué de la truncación en GF(251) y la naturaleza exacta del paso de permutación que Wu y Lo reemplazan por XOR.

Debilidades de organización:

- No hay análisis de complejidad ni comparación cuantitativa contra Thien-Lin (PSNR de las imágenes recuperadas, tasa de bloques perturbados, tamaños de sombra).
- La sección *Experimental Results* se reduce a cuatro figuras (secreto, randomizado, sombras, recuperado) sin métricas asociadas.
- No se propone un modelo de adversario formal; el análisis de seguridad se limita a la probabilidad combinatoria de adivinar con $r - 1$ sombras y omite ataques estadísticos o de colusión.
- La esteganografía en portadoras — requisito del TP — no es tratada por el paper. Las sombras se asumen almacenables como archivos *noise-like*; el paso de embebido en imágenes anfitrionas es agregado íntegramente por la consigna.

Descripción de los algoritmos

El algoritmo de *sharing* (pasos 1–8) es claro en su estructura: XOR con tabla aleatoria R , bucle por bloques de r píxeles que forman los coeficientes del polinomio, evaluación en $x = 1, \dots, n$, y mitigación si $f_j(x) = 256$. El punto más débil es el paso 5 (mitigación): la prescripción “*let the first non-zero pixel of $\{a_0, a_1, a_2, \dots, a_{n-1}\}$ be decreased by 1*” contiene un **error tipográfico** — el polinomio tiene r coeficientes, así que el conjunto debería ser $\{a_0, \dots, a_{r-1}\}$. Además, el paso omite especificar qué hacer si el decremento lleva un coeficiente a -1 , que en GF(257) es congruente con 256, precisamente el valor que se está intentando evitar. El implementador debe

inferir que la regla sólo aplica a coeficientes en el rango $[1, 255]$; el paso 5 indica “first non-zero” justamente para evitar esta patología.

El algoritmo de *reveal* (pasos 1–6) es correcto pero de alto nivel: “*use Lagrange’s interpolation to solve for the coefficients $a_0, \dots, a_{r-1}$* ” no especifica cómo hacerlo eficientemente. La consigna llena este vacío recomendando la variante *reducida* de Lagrange, evaluada en $x = 0$ y aplicada iterativamente para reducir el grado del polinomio en cada paso. Otra omisión importante: el paper asume que el receptor conoce qué abscisa x_i corresponde a cada sombra, sin especificar cómo se transmite esa información. La consigna resuelve esto mandando que el índice se grabe en los bytes 8–9 de cada portadora.

Notación

La notación es internamente consistente con la excepción del paso 5 ya mencionada. Otros detalles menores que dificultan la lectura estricta:

- Alterna entre *Thein-Lin* y *Thien-Lin* para referirse al paper de 2002 (la grafía correcta es *Thien-Lin*). El desliz aparece tanto en el cuerpo como en las referencias.
- Usa $f_j(x)$ para el polinomio de la sección j , pero al citar a Thien-Lin lo llama $q_j(x)$. No genera confusión dentro de un mismo paper, pero quien cruza ambos textos debe traducir mentalmente.
- La tabla aleatoria R se introduce en el paso 1 sin definir tamaño, rango ni algoritmo de generación; sólo se indica que es “*generated by a pseudo random number generator in advance*”. La consigna pone reglas precisas mediante el anexo *Tabla de Permutación* (LCG de 48 bits con semilla de 16 bits), pero alguien siguiendo únicamente el paper no podría implementar el esquema de manera interoperable.
- La demostración de que no existe la condición de todos los coeficientes en cero cuando $f_j(x) = 256$ escribe $a_0 = a_1 = a_2 = a_{r-1} = 0$ sin elipsis intermedia, sugiriendo un caso particular ($r = 4$) presentado como general.

En conclusión: la notación es aceptable para una lectura general pero presenta inconsistencias menores y omisiones que complican una implementación estrictamente fiel al texto.

2. ¿La imagen recuperada es idéntica a la original?

Consigna 5.2. La imagen recuperada, ¿es exactamente igual a la original ocultada? ¿Por qué?

No, no es exactamente igual. La pérdida proviene de una decisión algorítmica deliberada del esquema de Wu y Lo, no de errores de redondeo ni de pérdida en la esteganografía LSB (que es bit-exacta en sí misma).

Origen del problema: el valor 256 en $\text{GF}(257)$

El esquema trabaja en $\text{GF}(257)$ porque 257 es el menor primo estrictamente mayor que 255, y permite así representar todos los valores de un byte (0–255) como elementos del cuerpo. La evaluación de un polinomio

$$f_j(x) = (a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1}) \text{ mód } 257,$$

con coeficientes $a_i \in [0, 255]$, produce un resultado en $[0, 256]$. El valor **256 no es representable en un byte**: si se almacenara, la asignación a un `uint8_t` colapsaría a 0, indistinguible del caso $f_j(x) = 0$ legítimo, y la recuperación posterior produciría coeficientes incorrectos para todo el bloque.

La regla de mitigación

Tanto Thien y Lin (en GF(251), truncando píxeles 251–255 a 250) como Wu y Lo (en GF(257)) optan por *modificar el polinomio* antes que tolerar el desbordamiento. Wu y Lo, en su paso 5, prescriben: “If $f_j(x) = 256$, let the first non-zero pixel of $\{a_0, \dots, a_{r-1}\}$ be decreased by 1, and goto Step 4”. Nuestra implementación adopta una variante equivalente en espíritu: cuando alguna evaluación $f_j(i)$ con $i \in \{1, \dots, n\}$ produce 256, se incrementa a_0 módulo 256 y se reintentan *todas* las evaluaciones del bloque hasta que ninguna colapse. El paper prueba que la condición de todos los coeficientes en cero es incompatible con $f_j(x) = 256$, garantizando terminación.

Por qué esto introduce pérdida

Cuando se perturba a_0 (o, en la prescripción del paper, el primer coeficiente no nulo), las sombras almacenadas corresponden ya al polinomio *modificado*, no al original. Durante la recuperación, la interpolación de Lagrange reconstruye exactamente los coeficientes del polinomio del cual provienen las sombras — es decir, los coeficientes modificados. La pérdida es por tanto **irreversible desde la fase de distribución**: no hay forma de saber, mirando las sombras, qué bloques fueron perturbados ni cuánto.

En términos cuantitativos, el peor caso para un bloque perturbado es que los k píxeles del bloque difieran del original por la magnitud de la perturbación (típicamente ± 1 por iteración, multiplicado por el número de iteraciones que el ciclo de mitigación haya necesitado). Para imágenes naturales el porcentaje de bloques afectados es bajo y la imagen recuperada es *visualmente* indistinguible de la original, pero el resultado nunca es bit-exacto.

Existencia de una versión sin pérdida

Thien y Lin [2] describen una versión *lossless* (sección 3.3 de [2]) que evita la pérdida al precio de un crecimiento de las sombras: cada píxel con valor ≥ 250 se codifica como el par $(250, p_i - 250)$ en un arreglo expandido. Wu y Lo omiten esta variante en su esquema GF(257) y aceptan la pérdida como trade-off por simplicidad y tamaño fijo de sombra. La consigna pide implementar el esquema de Wu y Lo, por lo que adoptamos la versión con pérdida; en la sección 7 se discute la portación al modo *lossless* como mejora posible.

Otras fuentes de no-identidad

Para evitar atribuir toda la pérdida a la regla del 256, vale aclarar qué *no* contribuye:

- La máscara XOR es bit-exactamente reversible (propiedad $A \oplus B \oplus B = A$).
- La esteganografía LSB *replacement* reemplaza exactamente b bits del byte portador y los recupera exactamente; no hay redondeo.
- La aritmética modular en GF(257) es exacta para todas las operaciones (suma, resta, multiplicación e inverso multiplicativo precomputado).

La regla del 256 es, por construcción del esquema Wu/Lo, la única fuente de pérdida.

3. Seguridad del almacenamiento de la semilla y el número de sombra

Consigna 5.3. Analizar la seguridad en la ubicación del número de sombra y de la semilla de permutación. ¿De qué otra manera o en qué otro lugar podrían guardarse?

El esquema actual

La consigna fija el almacenamiento de dos metadatos en bytes reservados de la cabecera BMP:

- **Bytes 6–7:** semilla del LCG de 16 bits, en little-endian.
- **Bytes 8–9:** número de sombra ($1 \leq x \leq n \leq 10$), también en little-endian.

Estos bytes corresponden a campos `bfReserved1` y `bfReserved2` de la cabecera `BITMAPFILEHEADER`, que el estándar BMP declara como “reservados, deben ser cero”. La mayoría de los visores los ignoran, por lo que reescribirlos no altera la apariencia ni el funcionamiento del archivo como imagen.

Análisis de seguridad

Riesgo 1: la semilla en claro debilita la máscara XOR. Si un atacante reúne k portadoras y conoce el formato del esquema (o lo deduce inspeccionando los bytes 6–7, que en BMPs normales son cero), puede leer la semilla directamente y reproducir bit a bit la secuencia pseudoaleatoria del LCG. Esto reduce la seguridad del componente de *permutación* a la seguridad puramente combinatoria del esquema (k, n) : cualquier subconjunto de tamaño k permite la reconstrucción completa.

Riesgo 2: identificación pasiva de portadoras. En BMPs no manipulados, los bytes 6–9 son típicamente 00 00 00 00. Cualquier archivo con esos bytes no nulos — particularmente con un patrón consistente en el byte 9 igual a 00 y el byte 8 en `[01, 0A]` — es identificable como portadora por un adversario que conozca el esquema, incluso sin extraer la sombra. Esto viola parcialmente el principio esteganográfico: la *existencia* del mensaje oculto deja de ser invisible.

Riesgo 3: el número de sombra no es secreto, pero es delator. El valor de x asociado a cada sombra es información que la recuperación necesita conocer (Lagrange evalúa en abscisas conocidas). No tiene valor criptográfico ocultarlo a un atacante que ya posee k sombras — de hecho, si lo tuviera oculto, debería probar todas las $\binom{n}{k} \cdot k!$ asignaciones posibles. La ubicación elegida es razonable desde el punto de vista funcional pero, como en el riesgo 2, contribuye a la identificación del archivo como portadora.

Lo que sí preserva el esquema. La propiedad fundamental del esquema (k, n) se mantiene: con menos de k sombras, ni la semilla ni el número de sombra alcanzan para reconstruir la imagen secreta. Wu y Lo demuestran que la probabilidad de adivinar el secreto con $k-1$ sombras es $(1/256)^{M/k}$, donde M es el número de píxeles. La semilla expuesta no degrada esa cota: la máscara XOR es una defensa en profundidad para evitar correlación entre píxeles vecinos, no la fuente primaria de seguridad.

Alternativas de almacenamiento

1. **Cifrar la semilla con una clave maestra fuera de banda.** Compartir entre los participantes una clave K por un canal independiente y almacenar en bytes 6–7 el valor $\text{Enc}_K(\text{semilla})$ (por ejemplo XOR con $K \bmod 2^{16}$, o AES truncado). Un atacante con las sombras pero sin K no puede invertir la máscara aunque junte las k portadoras. Costo: agrega un secreto compartido fuera de banda que el esquema (k, n) original no requería.
2. **Distribuir la semilla en LSBs de píxeles fijos.** En lugar de los bytes 6–7, ocultar los 16 bits de la semilla en los LSBs de 16 píxeles en posiciones predeterminadas (por ejemplo, los píxeles 0, 50, 100, ...). Esto la hace indistinguible del resto del payload esteganográfico y elimina la firma reconocible en la cabecera. Costo: complica la lectura y requiere que las posiciones no colisionen con bits de sombra.
3. **Derivar la semilla del conjunto de portadoras.** Usar $\text{seed} = H(\text{shadow}_1 \parallel \dots \parallel \text{shadow}_k) \bmod 2^{16}$ con H una función hash. La semilla pasa a ser implícita: emerge sólo cuando se reúnen las k sombras correctas. Inconveniente: la mitigación del 256 puede modificar sombras durante la

distribución, así que el hash debe calcularse después de fijar el polinomio, lo que complica el proceso iterativo.

4. **Dispersar el número de sombra en múltiples campos inocuos.** En vez de bytes 8–9, codificar x como una función (por ejemplo, paridad) de varios bytes del color table o del padding de filas. Más difícil de detectar pasivamente, pero frágil ante visores que normalicen el archivo al reabrirlo.

La elección de ITBA — bytes reservados en claro — prioriza simplicidad y compatibilidad con visores estándar. Es defendible bajo el supuesto de que la seguridad del esquema (k, n) no depende del secreto de la semilla, sino del umbral. Para usos reales fuera del contexto académico se recomendaría la alternativa 1 (cifrado de la semilla con clave fuera de banda).

4. Criterio para $k \neq 8$: tamaño de portadoras y método de ocultamiento

Consigna 5.4. Explicar el criterio utilizado para ocultar las sombras en el caso de k distinto de 8. ¿De qué tamaño deberán ser las portadoras? Indicar si hubo propuestas previas que fueron descartadas.

Restricción geométrica

Sea M el número de píxeles de la imagen secreta. El esquema produce n sombras, cada una de tamaño

$$L_s = \lceil M/k \rceil \text{ bytes.}$$

Para $k = 8$, $L_s = M/8$, y con LSB1 (un bit por byte de portadora) hace falta $8 \cdot L_s = M$ bytes de portadora. Resultado: portadora del mismo tamaño que el secreto. Caso natural y previsto en la consigna.

Para $k \neq 8$, los L_s bytes de sombra requieren $L_s \cdot (8/b)$ bytes de portadora, donde b es el número de bits LSB utilizados por byte. Se desea que esto sea $\leq M$ para reusar portadoras de igual dimensión que el secreto.

Criterio adoptado: portadora del mismo tamaño, b variable

Asignamos el número de bits LSB en función de k de modo que $L_s \cdot (8/b) \leq M$ se cumpla con margen suficiente:

k	Bits LSB (b)	L_s	Bytes de portadora consumidos
2 – 3	4	$\lceil M/k \rceil$	$2\lceil M/k \rceil \leq M$
4 – 7	2	$\lceil M/k \rceil$	$4\lceil M/k \rceil \leq M$
8 – 10	1	$\lceil M/k \rceil$	$8\lceil M/k \rceil \leq M$

En todos los casos la portadora es del mismo tamaño que la imagen secreta. La consecuencia operativa es uniforme: el directorio de portadoras se valida exigiendo dimensiones iguales a la secreta para *cualquier* k , lo que simplifica el flujo y — al copiar la cabecera de una portadora durante la recuperación — garantiza que la imagen recuperada conserve dimensiones correctas sin necesidad de metadatos adicionales sobre el tamaño original.

Propuesta previa descartada: portadoras de tamaño M/k

Una alternativa que se evaluó es generar *portadoras más chicas*, de exactamente L_s píxeles (M/k bytes), usando LSB1 fijo. Tiene la ventaja teórica de ser más eficiente en almacenamiento total, pero fue descartada por varias razones:

1. **Rompe la uniformidad de portadoras.** Para cada k las portadoras tendrían dimensiones diferentes, exigiendo al usuario mantener pools de portadoras dimensionadas para cada k posible. No es práctico.
2. **Dificulta la recuperación del tamaño original.** Si la portadora no comparte dimensiones con la secreta, la recuperación necesita conocer las dimensiones de la secreta por otra vía (más metadatos en la cabecera, o un archivo lateral). El esquema $(8, n)$ explota la coincidencia de tamaños justamente para evitar este problema; querríamos preservar esa propiedad para todos los k .
3. **Compromete el camuflaje esteganográfico.** El espíritu de la esteganografía es que la portadora se vea como una imagen común. Imágenes de tamaños no estándar (por ejemplo $M/3$ píxeles para $k = 3$) son inmediatamente sospechosas.
4. **No aprovecha la capacidad disponible.** Una portadora del mismo tamaño que la secreta tiene M bytes; usar sólo M/k de ellos desperdicia más del 50 % del espacio para $k \geq 2$.

Trade-off de calidad visual en función de b

El método *LSB replacement* con b bits introduce un error máximo de $2^b - 1$ por píxel de portadora:

- $b = 1$ ($k \geq 8$): error máximo ± 1 . Imperceptible.
- $b = 2$ ($k \in [4, 7]$): error máximo ± 3 . Difícil de detectar visualmente, comparable al *noise floor* de imágenes fotográficas.
- $b = 4$ ($k \in [2, 3]$): error máximo ± 15 . Visualmente perceptible en zonas suaves (gradientes, cielo), aunque tolerable para portadoras con textura.

La degradación visual en $k = 2, 3$ es el precio a pagar por preservar uniformidad de tamaño y simplicidad del flujo. Para usos donde la calidad visual de las portadoras sea crítica, una alternativa sería forzar $k \geq 4$ o aceptar portadoras de tamaño doble para $k = 2, 3$ con LSB2.

5. Aspectos del algoritmo implementado

5.1. Facilidad de implementación

Consigna 5.5. Discutir la facilidad de implementación del algoritmo.

La implementación divide cómodamente en partes triviales y partes delicadas.

Partes triviales.

- *Aritmética en $\text{GF}(257)$.* La tabla de inversos se precomputa por fuerza bruta en $O(p^2)$ una sola vez al arranque y cabe holgadamente en memoria (257 entradas). Suma, resta y multiplicación son operaciones de una línea con un `% 257` bien colocado (`src/math_gf257.c`, ~50 líneas).
- *LCG de 48 bits.* Cuatro líneas copiadas literalmente del anexo de la consigna. Auto-verificable mediante un test de determinismo (misma semilla $\Rightarrow$ misma secuencia).
- *Embebido y extracción LSB.* Shifts y máscaras de bits, parametrizable sobre 1, 2 o 4 bits.
- *Parseo CLI y evaluación polinómica por Horner.* Rutinario.

Partes delicadas.

- **Manejo del formato BMP.** El *pixel offset* es dinámico: 54 para BMPs mínimos, pero 1078 para BMPs de 8 bpp con tabla de colores completa ($54 + 256 \cdot 4$). Asumir 54 funciona en pruebas casuales y falla en producción. Adicionalmente: padding de filas a múltiplos de 4 bytes, altura potencialmente negativa (bottom-up vs top-down) y endianness en los campos del encabezado.
- **Lagrange reducido numérico.** Conceptualmente simple — recuperar a_0 , restarlo de cada y_i , dividir por x_i , recursar sobre un polinomio de grado menor — pero con muchos lugares para errores sutiles: orden de la sustracción modular, división modular vía inverso precomputado, indexación al reducir.
- **Bucle de mitigación del 256.** Tras cada perturbación hay que reevaluar *todas* las abscisas $x \in \{1, \dots, n\}$, no sólo la que falló. Es el error más frecuente al implementar.
- **Endianness al persistir metadatos.** La consigna ilustra que `seed = 641` se almacena como `[0x81, 0x02]` (little-endian). El sentido es obvio una vez visto el ejemplo, pero implementarlo mal silenciosamente produce sombras que el receptor no puede leer.

Sobre el testing. En ausencia de archivos de referencia de la cátedra durante el desarrollo, la única señal de corrección disponible es el round-trip end-to-end del propio programa. Un round-trip exitoso no garantiza interoperabilidad: bugs simétricos pueden quedar enmascarados (por ejemplo, escribir endianness al revés y leerlo al revés produce un test verde pero archivos incompatibles con cualquier otra implementación). Por eso la implementación se acompaña de tests unitarios por componente (`tests/test_math_permutation.c`, `tests/test_distribute.c`, `tests/test_bmp.c`) y debe validarse, en último término, contra los archivos definitivos provistos por la cátedra.

Esfuerzo estimado. 40–60 horas-persona para una implementación limpia en C99 con tests, asumiendo familiaridad previa con manipulación de bits y aritmética modular.

5.2. Extensión a imágenes en color de 24 bpp

Consigna 5.5. Discutir la posibilidad de extender el algoritmo para que se usen imágenes en color (de 24 bits por píxel).

El esquema implementado opera sobre buffers de bytes, donde cada byte se interpreta como un coeficiente de polinomio en $\text{GF}(257)$. Las imágenes BMP de 24 bpp almacenan 3 bytes por píxel en orden B, G, R. Hay tres estrategias plausibles para extenderlo, con distintos compromisos:

Estrategia 1: separación por plano de color (recomendada). Tratar el archivo 24 bpp como tres imágenes 8 bpp superpuestas (I_B, I_G, I_R). Cada plano se distribuye independientemente con el mismo esquema (k, n) , generando tres juegos de sombras. Cada portadora 24 bpp aloja, en sus tres planos, las sombras correspondientes. Ventajas:

- Reutiliza por completo el motor existente; sólo cambia el código de lectura/escritura BMP y el bucle externo.
 - La capacidad de la portadora se triplica ($M \rightarrow 3M$ bytes), permitiendo usar LSB1 incluso para k pequeño y mejorando la calidad visual de las portadoras.
 - La semilla puede compartirse entre planos o usarse una distinta por plano, según se prefiera.
- Desventaja: el coste de almacenamiento es $3\times$ y la mitigación del 256 actúa independientemente sobre cada plano, lo que puede introducir ligeros shifts de color en los bloques perturbados.

Estrategia 2: tratar el archivo como un buffer plano. Procesar los $3M$ bytes del cuerpo BMP como un único stream de bytes, indistinguiendo planos. Implementación prácticamente sin cambios: sólo es necesario contemplar el offset y el padding por fila del formato 24 bpp (cada fila se alinea a múltiplo de 4 bytes). El esquema funciona matemáticamente, pero la mitigación del

256 puede afectar coordenadas de color individuales y degradar la fidelidad cromática del bloque más que la de luminancia.

Estrategia 3: agregar los 3 bytes en un único símbolo. Tratar cada píxel RGB como un valor de 24 bits y trabajar en $\text{GF}(p)$ con p primo mayor que 2^{24} (por ejemplo $p = 2^{24} + 43$). *No es práctico*: la tabla de inversos requeriría $\sim 16\text{M}$ entradas, la mitigación del overflow se vuelve más cara y se rompe el paradigma byte-a-byte del paper.

Recomendación. La estrategia 1 es la portación natural del algoritmo y la que aprovecha mejor la capacidad de las portadoras 24 bpp. Los cambios al código se limitan a: (i) parser BMP que detecte 24 bpp y maneje el padding de filas; (ii) split de la imagen secreta en tres planos antes de la fase de distribución; (iii) merge equivalente en la recuperación.

6. Dificultades en la lectura del paper e implementación

Consigna 5.6. ¿Qué dificultades tuvieron en la lectura del documento y/o en la implementación?

Las dificultades concretas que enfrentamos se agrupan en tres categorías.

Lectura del paper.

- El paper no es autocontenido. Requiere consultar permanentemente a Thien-Lin [2] para entender decisiones que Wu y Lo dan por supuestas: la motivación de la truncación en $\text{GF}(251)$, la naturaleza de la permutación que reemplazan por XOR, el esquema $1/r$ de compresión de sombras.
- El error tipográfico del paso 5 (índice a_{n-1} en vez de a_{r-1}) costó tiempo de comprensión: en un primer intento asumimos que el paper era correcto y construimos una versión del algoritmo donde el polinomio tenía n coeficientes en vez de r , lo cual no cierra con el resto del texto.
- Ambigüedad sobre cómo proceder si *todos* los coeficientes del bloque están en cero cuando se detecta $f_j(x) = 256$. El paper demuestra que esa condición es imposible, pero la demostración aparece al pie de la descripción del paso 5 y es fácil pasarla por alto al implementar.
- La tabla aleatoria R no está definida formalmente en el paper. La consigna la suple con el anexo de Tabla de Permutación, pero un lector que sólo tenga el paper a la vista no puede producir una implementación interoperable.

Implementación.

- Detectar el *pixel offset* dinámico fue el primer escollo. Las pruebas iniciales con BMPs sintéticos (offset fijo en 54) ocultaron el problema hasta que probamos con los assets de la cátedra, que vienen con offset 1078.
- Lagrange reducido tomó varios intentos. El error típico fue invertir el orden de la sustracción modular ($y_i - a_j$ vs $a_j - y_i$): los signos en $\text{GF}(257)$ no producen warnings obvios, sólo coeficientes mal recuperados, y el síntoma — imagen recuperada con ruido — no apunta a ninguna línea específica de código.
- La primera versión del bucle de mitigación del 256 sólo reevaluaba la abscisa que había fallado, no las n . El round-trip pasaba con cierta frecuencia y fallaba con otra hasta que aislamos el patrón.
- Decidir el tratamiento de $k \neq 8$ sin guía explícita del paper requirió análisis propio (ver §4).

Testing y validación.

- Hasta la entrega de los archivos definitivos por parte de la cátedra, la única señal de corrección era la autoconsistencia del round-trip. Confiar exclusivamente en eso es arriesgado: bugs simétricos en serialización (endianness) pueden hacer pasar todos los tests propios y aún producir archivos incompatibles con el evaluador.

- Durante las pruebas finales en el servidor pampero detectamos un caso anómalo: en el round-trip ciertos píxeles del bloque perturbado divergen del original por hasta 195/255, cuando la teoría predice desviaciones de unas pocas unidades por iteración de mitigación. El caso queda documentado para investigación posterior.

7. Extensiones y modificaciones propuestas

Consigna 5.7. ¿Qué extensiones o modificaciones harían a la implementación o al algoritmo?

A partir de la implementación realizada, identificamos las siguientes extensiones de valor:

1. **Versión *lossless*.** Portar al modo sin pérdida de Thien y Lin (sección 3.3 de [2]): codificar píxeles ≥ 250 como pares $(250, p_i - 250)$ en un arreglo expandido E antes de la fase polinómica. La sombra crece ligeramente (proporcional a la fracción de píxeles claros) pero la recuperación pasa a ser bit-exacta. Es la única manera de eliminar la pérdida discutida en la sección 2 sin abandonar la estructura del paper.
2. **Soporte para imágenes en color (24 bpp).** Extensión por separación de planos descrita en la sección 5.2.
3. **Cifrado de la semilla con clave fuera de banda.** Alternativa 1 de la sección 3: agregar una clave maestra simétrica compartida entre los n depositarios, y almacenar $\text{Enc}_K(\text{semilla})$ en lugar de la semilla en claro. Defensa en profundidad contra adversarios que obtengan k portadoras pero no la clave maestra.
4. **Verificación de integridad por sombra.** Agregar al payload de cada sombra un CRC32 o un MAC corto calculado sobre los bytes de sombra. Permite detectar portadoras corruptas o adulteradas *antes* de presentar la imagen reconstruida, en vez de mostrar ruido inconducente.
5. **Esquemas verificables (Verifiable Secret Sharing).** Incorporar compromisos de Feldman o Pedersen sobre los coeficientes del polinomio para que cada depositario pueda verificar la consistencia de su sombra con el resto, sin necesidad de confiar en el *dealer* ni revelarla prematuramente.
6. **Distribución progresiva.** Siguiendo la línea de Chen y Lin [4], ordenar las sombras por importancia (frecuencia visual, energía espectral) y permitir reconstrucción incremental con calidad creciente conforme se aportan más sombras.
7. **Endurecimiento esteganográfico.** La opción de almacenar la semilla en LSBs de píxeles predeterminados (alternativa 2 de la sección 3) eliminaría la firma identificable en bytes reservados de la cabecera y mejoraría la *indetectabilidad* estadística de las portadoras.
8. **Cuadro de inversos parametrizado.** La tabla de inversos de $\text{GF}(257)$ se precomputa en $O(p^2)$ por fuerza bruta; una implementación con algoritmo extendido de Euclides la calcularía en $O(p \log p)$. Ahorro marginal en este caso (257 elementos), pero relevante si se experimenta con primos más grandes.

8. Aplicaciones del esquema

Consigna 5.8. ¿En qué situaciones aplicarían este tipo de algoritmos?

El esquema (k, n) de secreto compartido sobre imágenes con esteganografía aplica en escenarios que cumplen al menos dos de tres condiciones: necesidad de *autorización plural* (m -de- n), tolerancia a fallas o pérdidas parciales (hasta $n - k$ custodios pueden desaparecer), y deseabilidad de que la *existencia* misma del secreto sea no evidente.

Custodia distribuida de claves criptográficas. Claves maestras de PKI, claves raíz de autoridades certificadoras, semillas de *cold wallets* de criptomonedas y *master keys* de HSMs se distribuyen entre múltiples custodios para que ningún individuo pueda recuperarlas y la pérdida de hasta $n - k$ custodios no comprometa la disponibilidad. La capa esteganográfica protege a los custodios contra ataques dirigidos: una sombra dentro de una foto familiar no llama la atención.

Backup distribuido sobre infraestructura inconfiable. Respaldos críticos sobre múltiples proveedores cloud (típicamente 3 de 5 entre AWS, GCP, Azure y otros) de modo que ningún proveedor accede al contenido completo y la caída simultánea de hasta $n - k$ proveedores no destruye el respaldo. Útil también para custodia en redes peer-to-peer sin coordinador central.

Comunicación encubierta con negación plausible. Periodistas, activistas o disidentes operando en redes vigiladas pueden enviar n imágenes aparentemente inocuas por canales separados. Un adversario que intercepte $r - 1$ de ellas no obtiene información alguna; un adversario que detecte la naturaleza esteganográfica de las imágenes debe reunir k para reconstruir el mensaje.

Custodia plural de información sensible. Imágenes médicas (estudios DICOM oncológicos o psiquiátricos), documentos legales sellados (testamentos, contratos con cláusulas de revelación diferida), evidencia forense con cadena de custodia distribuida: casos donde la revelación requiere coordinación explícita entre múltiples partes y la trazabilidad de la decisión de revelar es valiosa.

Aplicaciones militares y de inteligencia. Mapas operacionales, posiciones de unidades y claves de comunicación táctica distribuidas entre n unidades de modo que la captura o pérdida de hasta $n - k$ no exponga la información, y la reconstrucción requiera coordinación explícita entre k unidades autorizadas.

Watermarking distribuido y robusto. Marcas de agua de autoría o identificadores de licencia embebidos como sombras en múltiples copias de una obra. Un atacante con $r - 1$ copias no puede removerlas; un usuario legítimo con k copias puede verificar la autoría.

Sistemas con compartimentalización fuerte. Aplicaciones que exigen el *n-eyes principle* — accesos a sistemas críticos, decisiones sobre activos de alto valor, ejecución de operaciones irreversibles — donde ningún individuo debe poder actuar solo.

En todos estos casos la combinación de (k, n) con esteganografía agrega valor sobre alternativas más simples (cifrado + replicación) porque ataca dos vectores de riesgo simultáneamente: la *confidencialidad* (un atacante con menos de k sombras no obtiene información) y la *indetectabilidad* (un atacante que observa el tráfico no identifica fácilmente cuáles archivos son portadores del secreto).

Referencias

- [1] K.-S. Wu, T.-M. Lo. “An Efficient Secret Image Sharing Scheme.” *Applied Mechanics and Materials*, vols. 284–287, pp. 3025–3029, 2013.
- [2] C.-C. Thien, J.-C. Lin. “Secret image sharing.” *Computers & Graphics*, vol. 26, pp. 765–770, 2002.
- [3] A. Shamir. “How to share a secret.” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [4] S.-K. Chen, J.-C. Lin. “Fault-Tolerant and Progressive Transmission of images.” *Pattern Recognition*, vol. 38, no. 12, pp. 2466–2471, 2005.